

SERVERLESS EC2 SCHEDULER

Automated EC2 Start/Stop using AWS Lambda, EventBridge and IAM

Project Documentation

Cloud Platform: Amazon Web Services (AWS)

Core Services: AWS Lambda, Amazon EC2, Amazon EventBridge, AWS IAM

Architecture: Serverless, event-driven automation

Targeting Method: EC2 resource tags

Purpose

Automatically stop development EC2 instances during non-working hours and start them again when work begins, reducing unnecessary compute costs.

1. Executive Summary

The Serverless EC2 Scheduler is an AWS automation project designed to control the operating state of selected Amazon EC2 instances on a scheduled basis. The solution uses Amazon EventBridge to trigger AWS Lambda functions at defined times. Lambda identifies EC2 instances dynamically using the `AutoSchedule=true` tag and then starts or stops those instances.

The project avoids hardcoding instance IDs and applies a least-privilege IAM policy so that the automation is intended to operate only on instances carrying the approved `AutoSchedule=true` tag. This makes the solution reusable when instances are added, replaced, or recreated.

2. Project Goal

The primary goals of the project are:

- Automate EC2 instance start and stop operations.
- Reduce unnecessary EC2 running time and associated compute costs.
- Avoid hardcoding EC2 instance IDs in Lambda code.
- Use EC2 tags for dynamic resource targeting.
- Apply least-privilege IAM permissions.
- Use EventBridge schedules for hands-free daily automation.
- Create a simple serverless architecture with no dedicated scheduler server.

3. Real-World Use Case

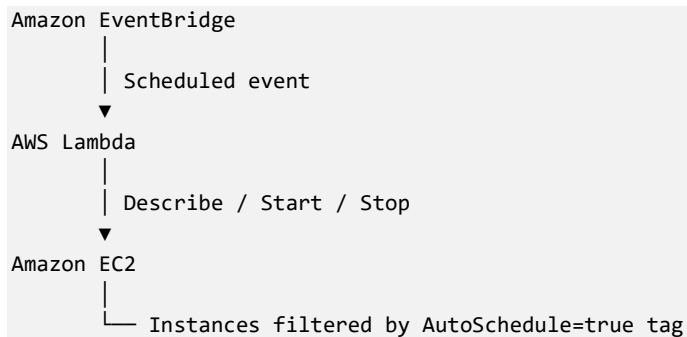
Development and testing servers are often required only during working hours. Leaving these instances running overnight or during weekends can create unnecessary costs. This project automates the routine:

- Morning: start tagged development instances so they are available for work.
- Evening: stop tagged development instances after the working day.

The same design can be adapted for test, staging, training, or other non-production workloads. Production instances should generally be excluded unless the scheduling policy has been carefully reviewed.

4. System Architecture

The architecture is:



Component responsibilities:

Component	Role	Purpose
Amazon EventBridge	Scheduler	Triggers Lambda at configured times.
AWS Lambda	Automation logic	Finds tagged instances and performs start/stop operations.
Amazon EC2	Target resources	Provides the compute instances being managed.
AWS IAM	Security	Controls exactly what Lambda is allowed to do.
EC2 Tags	Resource selection	Identifies which instances are managed by the scheduler.

5. Technology Stack

- Amazon EC2 — virtual servers to be scheduled.
- AWS Lambda — serverless execution environment.
- Amazon EventBridge — scheduled event generation.
- AWS IAM — role and permission management.
- Python — Lambda implementation language.
- Boto3 — AWS SDK used by Python Lambda code.
- EC2 Tags — dynamic instance targeting.

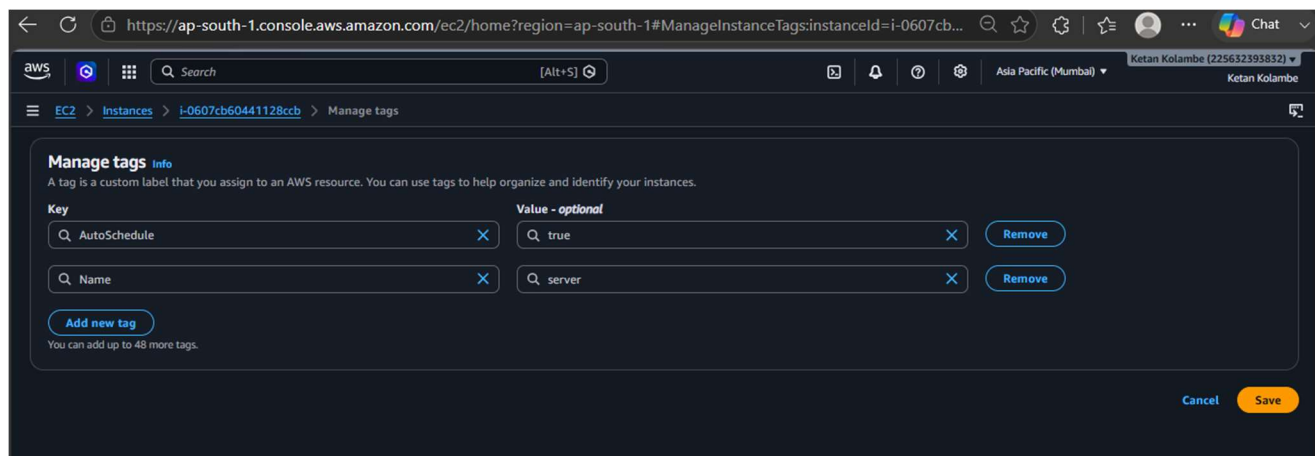
6. Implementation

6.1 Tag the EC2 Instance

Add the following tag to every EC2 instance that should be managed:

```
Key      = AutoSchedule
Value    = true
```

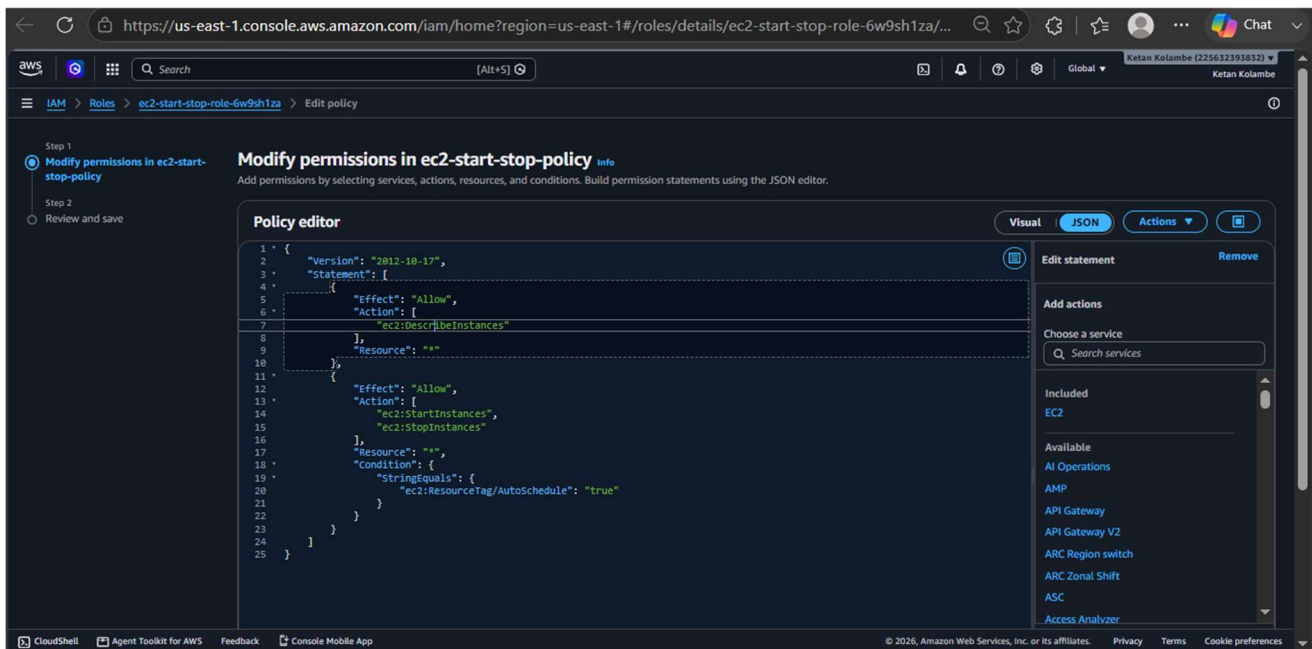
The tag acts as the selection mechanism. Any instance without this tag is not intended to be managed by the scheduler.



6.2 Create the Lambda IAM Role

Create an IAM execution role for Lambda and attach a custom policy containing only the permissions required by the function.

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Effect": "Allow",
      "Action": [
        "ec2:DescribeInstances"
      ],
      "Resource": "*"
    },
    {
      "Effect": "Allow",
      "Action": [
        "ec2:StartInstances",
        "ec2:StopInstances"
      ],
      "Resource": "*",
      "Condition": {
        "StringEquals": {
          "ec2:ResourceTag/AutoSchedule": "true"
        }
      }
    }
  ]
}
```



6.3 Create the Lambda Function

The Lambda function receives an action value of start or stop, discovers instances with the required tag, and performs the requested operation.

```

import boto3

ec2 = boto3.client('ec2')

def lambda_handler(event, context):
    action = event.get("action")

    response = ec2.describe_instances(
        Filters=[
            {
                'Name': 'tag:AutoSchedule',
                'Values': ['true']
            }
        ]
    )

    instance_ids = []

    for reservation in response['Reservations']:
        for instance in reservation['Instances']:
            instance_ids.append(instance['InstanceId'])

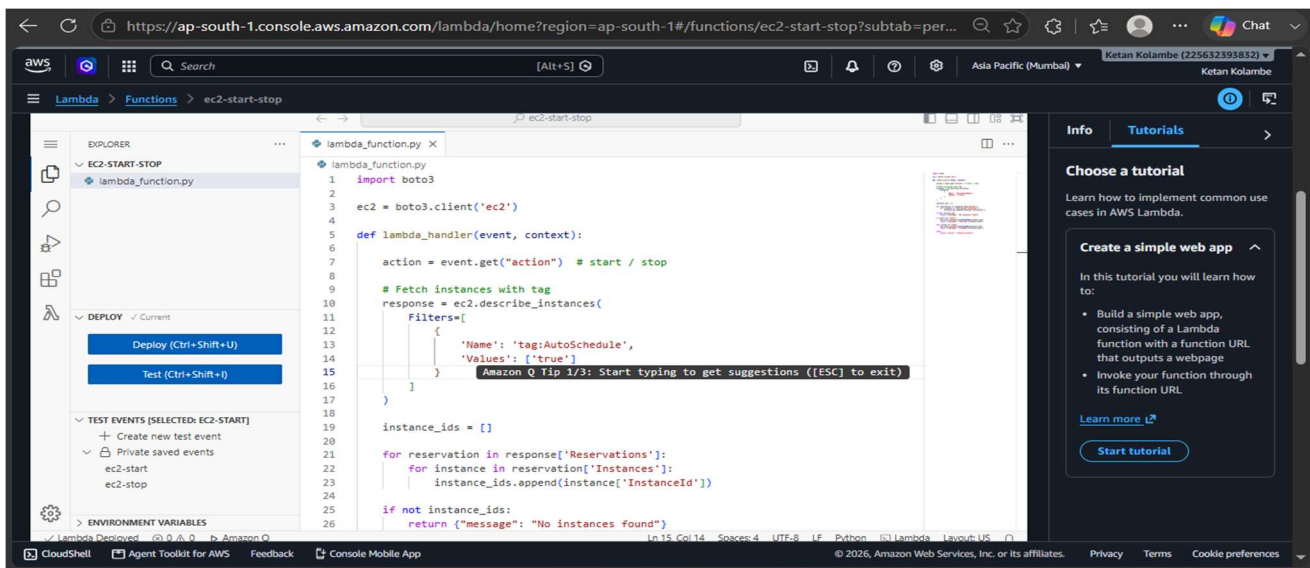
    if not instance_ids:
        return {"message": "No instances found"}

    if action == "start":
        ec2.start_instances(InstanceIds=instance_ids)
        return {"message": f"Started {instance_ids}"}

    elif action == "stop":
        ec2.stop_instances(InstanceIds=instance_ids)
        return {"message": f"Stopped {instance_ids}"}

    else:
        return {"error": "Invalid action"}

```



6.4 Create Lambda Test Event

Test the function manually before attaching the schedule.

Start test event:

```
{
  "action": "start"
}
```

Test event Info Delete CloudWatch Logs Live Tail Save Test

To invoke your function without saving an event, modify the event, then choose Test. Lambda uses the modified event to invoke your function, but does not overwrite the original event until you choose Save.

Test event action

☐ Create new event ☒ Edit saved event

Invocation type

☐ Synchronous
Executes the Lambda function and blocks until receiving the function's response, with a maximum timeout of 15 minutes. Returns function output or error details directly to the calling application.

☒ Asynchronous
Enqueues the Lambda function for execution and returns immediately with a request ID. Function processes independently, with results optionally sent to a configured destination like SQS, SNS, or EventBridge.

Event name

ec2-start

Event JSON Format JSON

```
1 {
2   "action": "start"
3 }
```

Stop test event:

```
{
  "action": "stop"
}
```

Test event Info Delete CloudWatch Logs Live Tail Save Test

To invoke your function without saving an event, modify the event, then choose Test. Lambda uses the modified event to invoke your function, but does not overwrite the original event until you choose Save.

Test event action

☐ Create new event ☒ Edit saved event

Invocation type

☐ Synchronous
Executes the Lambda function and blocks until receiving the function's response, with a maximum timeout of 15 minutes. Returns function output or error details directly to the calling application.

☒ Asynchronous
Enqueues the Lambda function for execution and returns immediately with a request ID. Function processes independently, with results optionally sent to a configured destination like SQS, SNS, or EventBridge.

Event name

ec2-stop

Event JSON Format JSON

```
1 {
2   "action": "stop"
3 }
```

6.5 Configure EventBridge Scheduler

EventBridge One-Time Schedule Configuration

- **Schedule Type:** One-time schedule
- **Target Execution Date:** 2026-08-20
- **Target Execution Time:** 01:26 (24-hour format)
- **Time Zone:** Asia/Calcutta (UTC+05:30)
- **Flexible Time Window:** Off (Mode: OFF)

Schedule pattern

Occurrence [Info](#)
You can define an one-time or recurrent schedule.

☒ One-time schedule ☐ Recurring schedule

Date and time
The date and time to invoke the target.

2026/08/20  01:26 (UTC+05:30) Asia/Calcutta
YYYY/MM/DD Use 24-hour format timestamp (hh:mm) Time zone

Flexible time window
If you choose a flexible time window, Scheduler invokes your schedule within the time window you specify. For example, if you choose 15 minutes, your schedule runs within 15 minutes after the schedule start time.

Off 

[Cancel](#) [Skip to Review and save schedule](#) [Next](#)

7. End-to-End Workflow

1. An EventBridge schedule reaches its configured execution time.
2. EventBridge invokes the Lambda function.
3. Lambda reads the action value: start or stop.
4. Lambda calls DescribeInstances and filters for AutoSchedule=true.
5. Lambda collects the matching EC2 instance IDs.
6. Lambda optionally checks the current instance state.
7. Lambda calls StartInstances or StopInstances.
8. Lambda returns the result and writes useful execution information to CloudTrail Logs.

Event history (50+) [Info](#)

Event history shows you the last 90 days of management events.

Lookup attributes

Read-only  Q: false  [Filter by date and time](#) [Clear filter](#) < 1 2 ... > 

☐	Event name	Event time	User name	Event source	Resource type	Resource name
☐	CreateLogStream	August 20, 2026, 01:26:21 (UTC...)	ec2-start-stop	logs.amazonaws.com	AWS::Logs::LogGroup	/aws/lambda/ec2-start-stop
☐	StopInstances	August 20, 2026, 01:26:18 (UTC...)	ec2-start-stop	ec2.amazonaws.com	AWS::EC2::Instance	i-0607cb60441128ccb
☐	UpdateSchedule	August 20, 2026, 01:24:41 (UTC...)	root	scheduler.amazonaws.com	AWS::Scheduler::Schedule	arn:aws:scheduler:ap-south-1:225632393832:schedule/default/ec2-start-stop
☐	StartInstances	August 20, 2026, 01:24:28 (UTC...)	root	ec2.amazonaws.com	AWS::EC2::Instance	i-0607cb60441128ccb
☐	CreateLogStream	August 20, 2026, 01:20:21 (UTC...)	ec2-start-stop	logs.amazonaws.com	AWS::Logs::LogGroup	/aws/lambda/ec2-start-stop
☐	StopInstances	August 20, 2026, 01:20:18 (UTC...)	ec2-start-stop	ec2.amazonaws.com	AWS::EC2::Instance	i-0607cb60441128ccb
☐	CreateSchedule	August 20, 2026, 01:14:18 (UTC...)	root	scheduler.amazonaws.com	AWS::Scheduler::Schedule	arn:aws:scheduler:ap-south-1:225632393832:schedule/default/ec2-start-stop
☐	CreateSchedule	August 20, 2026, 01:14:15 (UTC...)	root	scheduler.amazonaws.com	-	-

8. Security and Least Privilege

Least privilege means granting a workload only the permissions necessary to perform its job. In this project, Lambda needs to discover the target instances and start or stop only the instances marked for automation.

- No EC2 instance IDs are embedded in the code.
- StartInstances and StopInstances are restricted using the AutoSchedule=true resource-tag condition.
- The Lambda role does not need broad administrative permissions such as ec2:*
- The automation is easier to audit because resource selection is visible through tags.
- CloudWatch logging can be used to review executions and failures.

Operational caution: IAM tag-based controls should be treated as a security boundary. Protect the ability to modify the AutoSchedule tag, because changing the tag can change which resources are eligible for automation.

9. Cost Optimization

The main cost-saving mechanism is reducing the number of hours that eligible EC2 instances remain in the running state. Lambda and EventBridge are serverless services and generally contribute very little cost for a small scheduler workload, subject to AWS pricing and account usage.

Example: if a development instance would otherwise run 24 hours per day but is stopped for 12 hours each night, its running time can be reduced by approximately 50% during those scheduled periods. Actual savings depend on instance type, region, storage, data transfer, and other AWS charges.

10. Logging and Monitoring

Add clear log messages such as:

```
print(f"Action: {action}, Instances: {instance_ids}")
```

Review Lambda execution logs in Amazon CloudWatch Logs. Useful information to record includes:

- Action requested.
- Number of instances discovered.
- Instance IDs acted upon.
- Instances skipped because their state did not require action.

11. Advantages of the Project

- Serverless — no scheduler server is required.
- Automated — start/stop operations happen on schedule.
- Dynamic — tags remove the need to update Lambda when instance IDs change.
- Secure — permissions can be limited using IAM and resource tags.
- Cost-aware — reduces unnecessary EC2 running time.
- Scalable — the same function can manage multiple tagged instances.

12. Project Outcome

The completed project demonstrates a practical cloud automation pattern: an EventBridge schedule triggers a serverless Lambda function, Lambda discovers the correct EC2 resources through tags, and IAM controls what actions the function can perform. The design is simple enough for a learning project while providing a foundation for more advanced production automation.

13. Conclusion

The Serverless EC2 Scheduler provides an effective way to automate non-production EC2 start/stop operations. By combining EventBridge, Lambda, IAM and EC2 tags, the project removes manual intervention, avoids hardcoded

resource IDs, improves security through least privilege, and can reduce unnecessary compute usage. The architecture can be extended with monitoring, alarms, additional tagging rules, time-zone-aware schedules, and more sophisticated cost-control policies.